Tree Methods II

Math 243

Michael Pearce

Outline

Last time, we...

- Started our unit on "Trees"
- Discussed regression trees as a non-parametric model

In today's class, we will...

- Investigate pruning algorithms for improving accuracy of regression trees
- Create and prune regression trees in R

Section 1

Pruning

The general tree algorithm

1. Starting with entire data set S, search *every* cut-off (c) of *every* predictor (X_k) to split S into two groups S_1 and S_2:

$$S_1 = \text{observations with } X_k \leq c$$
$$S_2 = \text{observations with } X_k > c$$

that minimizes sum of squared error:

$$\text{SSE} = \sum_{i \in S_1} (y_i - \bar{y}_1)^2 + \sum_{i \in S_2} (y_i - \bar{y}_2)^2$$

where $\bar{y}_1$ and $\bar{y}_2$ are the average response value in groups S_1 and S_2

The general tree algorithm

1. Starting with entire data set S, search *every* cut-off (c) of *every* predictor (X_k) to split S into two groups S_1 and S_2:

$$S_1 = \text{observations with } X_k \leq c$$
$$S_2 = \text{observations with } X_k > c$$

that minimizes sum of squared error:

$$\text{SSE} = \sum_{i \in S_1} (y_i - \bar{y}_1)^2 + \sum_{i \in S_2} (y_i - \bar{y}_2)^2$$

where $\bar{y}_1$ and $\bar{y}_2$ are the average response value in groups S_1 and S_2

2. Repeat Step 1 on *both* S_1 and S_2 to make new groups.

The general tree algorithm

1. Starting with entire data set S, search *every* cut-off (c) of *every* predictor (X_k) to split S into two groups S_1 and S_2:

$$S_1 = \text{observations with } X_k \leq c$$
$$S_2 = \text{observations with } X_k > c$$

that minimizes sum of squared error:

$$\text{SSE} = \sum_{i \in S_1} (y_i - \bar{y}_1)^2 + \sum_{i \in S_2} (y_i - \bar{y}_2)^2$$

where $\bar{y}_1$ and $\bar{y}_2$ are the average response value in groups S_1 and S_2

2. Repeat Step 1 on *both* S_1 and S_2 to make new groups.

3. Repeat Step 1 on those new groups

The general tree algorithm

1. Starting with entire data set S, search *every* cut-off (c) of *every* predictor (X_k) to split S into two groups S_1 and S_2:

$$S_1 = \text{observations with } X_k \leq c$$
$$S_2 = \text{observations with } X_k > c$$

that minimizes sum of squared error:

$$\text{SSE} = \sum_{i \in S_1} (y_i - \bar{y}_1)^2 + \sum_{i \in S_2} (y_i - \bar{y}_2)^2$$

where $\bar{y}_1$ and $\bar{y}_2$ are the average response value in groups S_1 and S_2

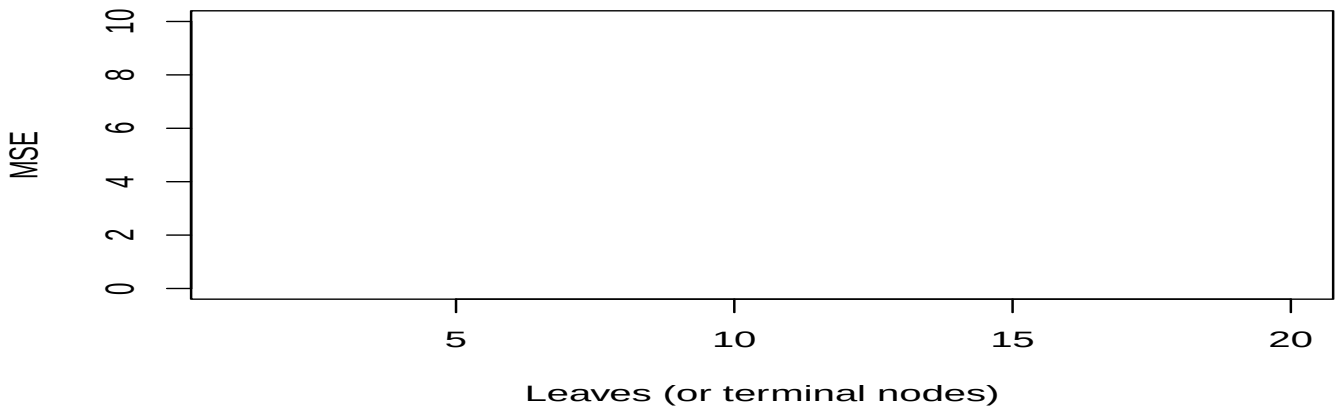
2. Repeat Step 1 on *both* S_1 and S_2 to make new groups.
3. Repeat Step 1 on those new groups
4. ...
5. Stop?

Big Question: How do we decide when to stop the algorithm?

Stopping the algorithm: Discussion Questions

Consider a plot of:

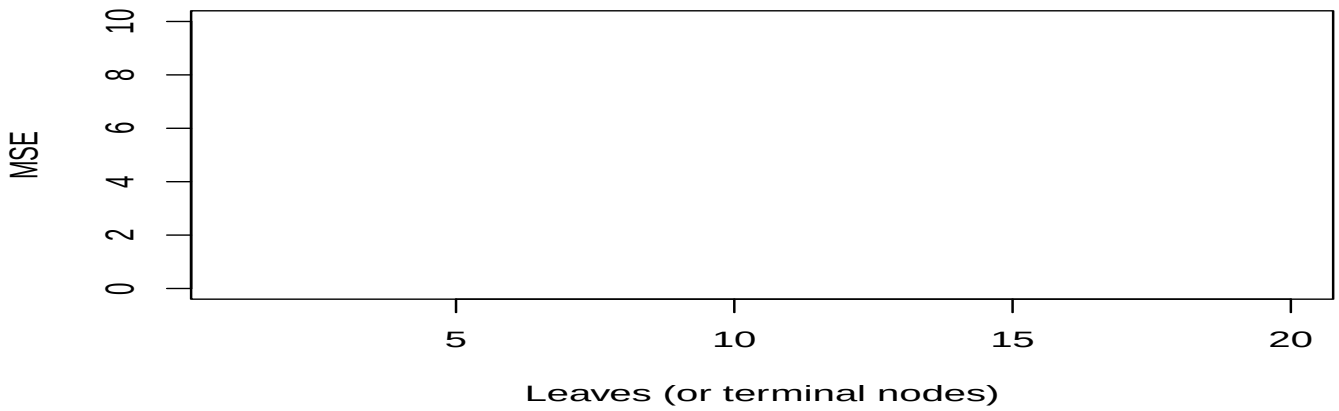
- **y axis:** training and testing MSE
- **x axis:** The number of leaves (or terminal nodes) of regression tree



Stopping the algorithm: Discussion Questions

Consider a plot of:

- **y axis:** training and testing MSE
- **x axis:** The number of leaves (or terminal nodes) of regression tree



Q: What would the *trends* for training and testing MSE look like in this plot? What can we relate the number leaves (or terminal nodes) to?

Q: The classic version of CART stops splitting the dataset once the groups have few enough (e.g., fewer than 5) observations, and then chooses that tree. What are some flaws of this approach?

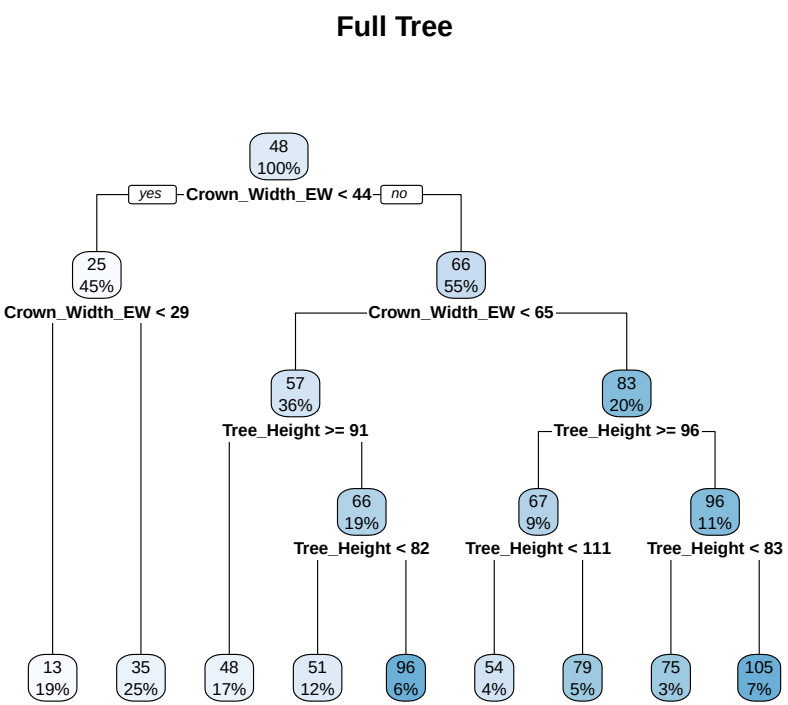
Subtrees

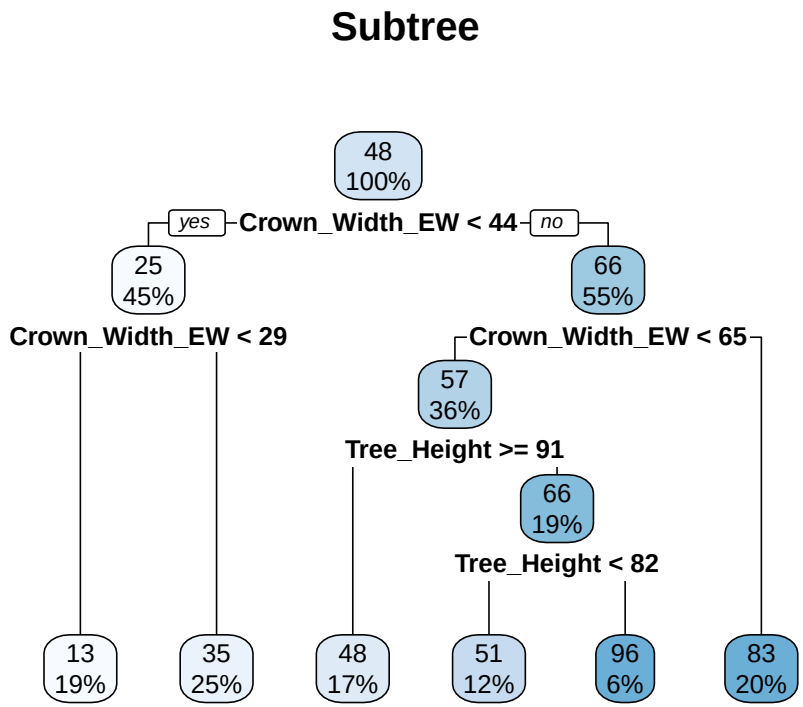
A **Subtree** is a regression tree obtained by removing some of the branches and nodes from the full regression tree.

Subtrees

A **Subtree** is a regression tree obtained by removing some of the branches and nodes from the full regression tree.

- For example:





- Q: What have I removed from the full tree to get the subtree above?

Best subtree?

An idea: We could compare CV MSE of *all* possible subtrees, and then pick the best subtree!

Best subtree?

An idea: We could compare CV MSE of *all* possible subtrees, and then pick the best subtree!

- This is *extremely* computationally expensive

Instead we use a faster process called Cost Complexity Pruning, aka, *weakest link pruning*.

Cost Complexity Pruning

Once a tree is fully grown, we can use **Cost Complexity Pruning** ("weakest link pruning").

Cost Complexity Pruning

Once a tree is fully grown, we can use **Cost Complexity Pruning** ("weakest link pruning").

Cost Complexity Pruning: Given:

- a *full* tree T_0
- a **tuning parameter** α

find the subtree T of the full tree T_0 that minimizes:

$$\mathrm{RSS} + \alpha |T|$$

where $|T|$ is the number of leaves or terminal nodes in the subtree T.

Cost Complexity Pruning

Once a tree is fully grown, we can use **Cost Complexity Pruning** ("weakest link pruning").

Cost Complexity Pruning: Given:

- a *full* tree T_0
- a **tuning parameter** α

find the subtree T of the full tree T_0 that minimizes:

$$\text{RSS} + \alpha|T|$$

where $|T|$ is the number of leaves or terminal nodes in the subtree T.

- Looks like LASSO regression!
- α penalizes trees with more leaves (or terminal nodes)
- As α increases, more and more leaves and branches are lost
- Nice Result: As α increases, the "weakest" branch in the tree is pruned away. This means the sequence of "best" subtrees are **nested**.

Pruning: Choosing α

Cross Validation for α: For a range of α values,

- Calculate the CV MSE of the pruned (sub)trees that correspond to each α
- Choose the α that gave the lowest CV MSE

Pruning: Choosing α

Cross Validation for α: For a range of α values,

- Calculate the CV MSE of the pruned (sub)trees that correspond to each α
- Choose the α that gave the lowest CV MSE

- Return to the full dataset, and fit the full tree
- Use the chosen cross-validated α to prune the tree

Section 2

Trees in R

Creating Tree Models in R

There are two common packages for creating regression trees in R: `tree` and `rpart`.

- ISLR uses `tree`.
- But we'll cover the `rpart` package here (newer, computationally faster, has more options, etc.).

Trees on Trees

We use a subset of the `pdxTrees` dataset from the `pdxTrees` package (maintained by K. McConville, I. Caldwell, and N. Horton)

- To keep things manageable, we'll focus on trees in 3 parks nearby Reed.

```
library(pdxTrees)
my_pdxTrees <- get_pdxTrees_parks(park = c("Powel Park", "Woodstock Park", "Berkeley Park"))
```

Trees on Trees

We use a subset of the `pdxTrees` dataset from the `pdxTrees` package (maintained by K. McConville, I. Caldwell, and N. Horton)

- To keep things manageable, we'll focus on trees in 3 parks nearby Reed.

```
library(pdxTrees)
my_pdxTrees <- get_pdxTrees_parks(park = c("Powel Park", "Woodstock Park", "Berkeley Park"))
```

- And use trees from another park as a test set:

```
my_pdxTrees_test <- get_pdxTrees_parks(park = c("Glenwood Park"))
```

Trees using rpart

Fitting a tree using predictors Tree_Height, Crown_Width_EW, Crown_Width_NS, and Crown_Base_Height:

```
set.seed(1)
library(rpart)
tree_model1 <- rpart(Carbon_Sequestration_lb ~
                       Tree_Height + Crown_Width_EW + Crown_Width_NS + Crown_Base_Height,
                     data = my_pdxTrees)
```

- Very similar to lm()

Trees using rpart

Fitting a tree using predictors Tree_Height, Crown_Width_EW, Crown_Width_NS, and Crown_Base_Height:

```
set.seed(1)
library(rpart)
tree_model1 <- rpart(Carbon_Sequestration_lb ~
                       Tree_Height + Crown_Width_EW + Crown_Width_NS + Crown_Base_Height,
                     data = my_pdxTrees)
```

- Very similar to lm()

Can change several features of the tree by adding a control argument:

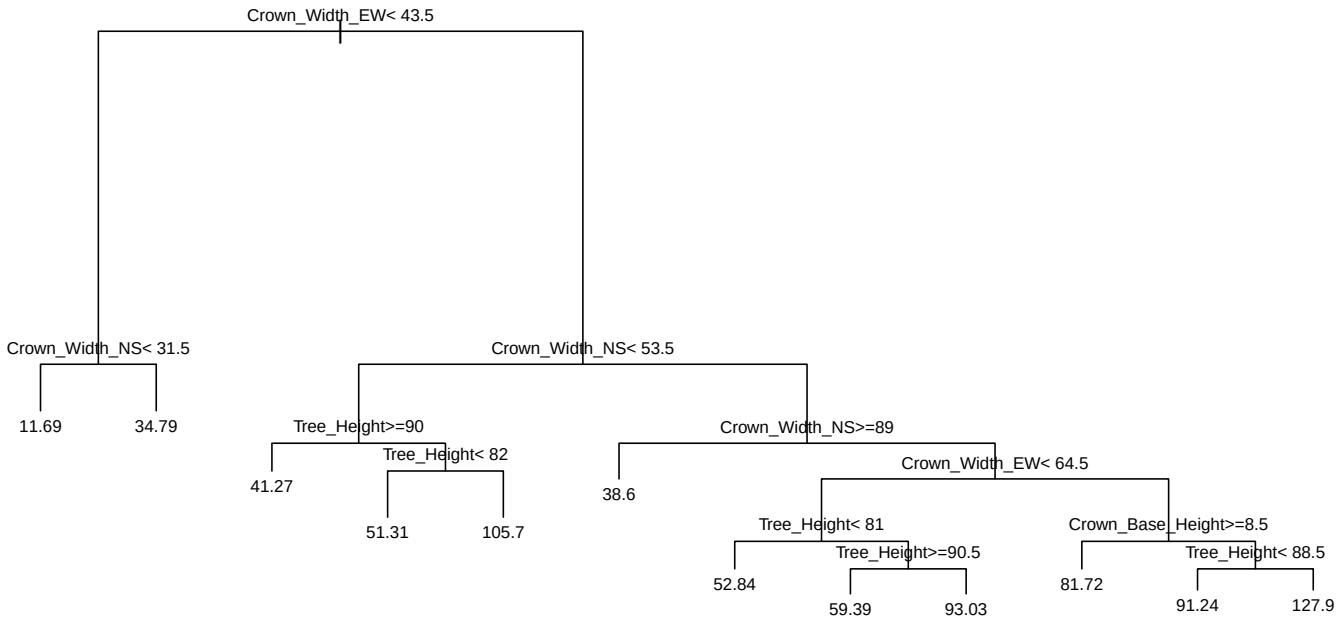
```
set.seed(1)
tree_model2 <- rpart(Carbon_Sequestration_lb ~
                       Tree_Height + Crown_Width_EW + Crown_Width_NS + Crown_Base_Height,
                     control = rpart.control(minsplit = 30, xval = 10, maxdepth = 8),
                     data = my_pdxTrees)
```

- minsplit is the minimum number of observations in a group
- xval is the number of cross-validation folds used
- maxdepth is the maximum depth of any node in the final tree

Plots using `plot`

The base R `plot` function quickly generates plots, but...

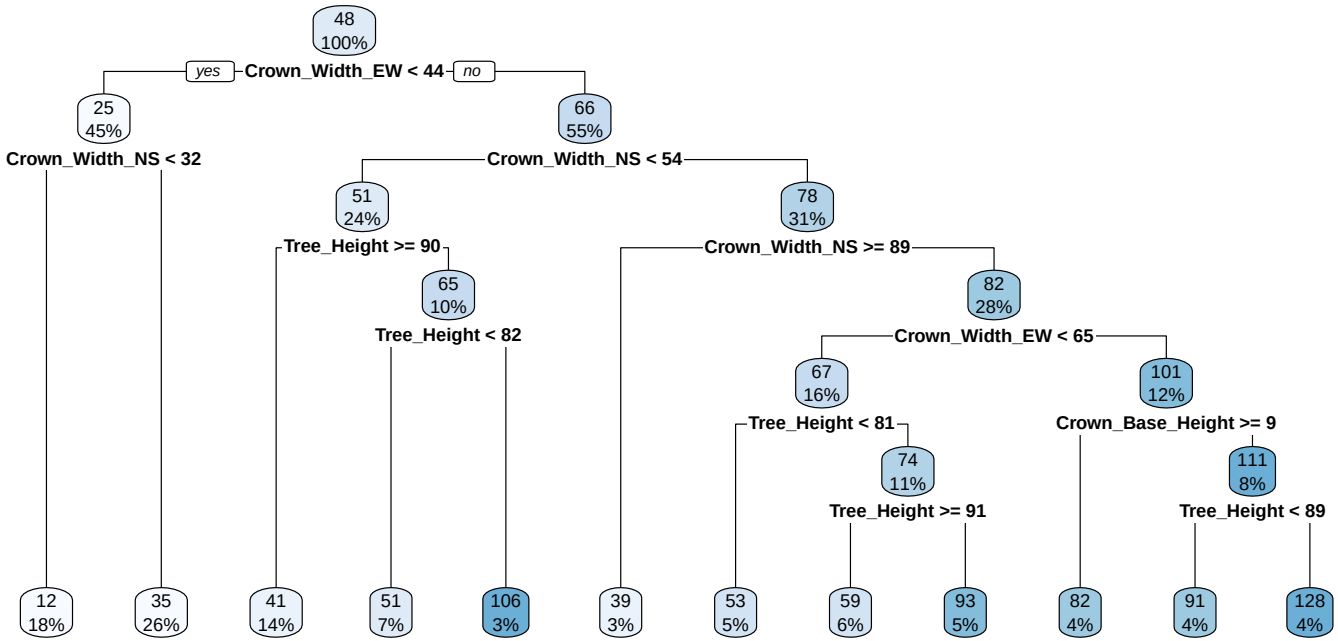
```
plot(tree_model1)
text(tree_model1, cex = .5)
```



Plots using `rpart.plot`

The `rpart.plot()` function from the `rpart.plot` package makes nicer ones:

```
library(rpart.plot)
rpart.plot(tree_model1)
```



- Some further customization available (see `?rpart.plot`)

Checking CV results in rpart

The rpart function automatically performs k-fold CV for the pruning parameter α

- To access results, append $cptable to the rpart model object:

```
tree_model1$cptable
```

```
##           CP nsplit rel error    xerror       xstd
## 1 0.31073097      0 1.0000000 1.0105895 0.09666964
## 2 0.07370105      1 0.6892690 0.7679112 0.07560215
## 3 0.04577064      2 0.6155680 0.7211540 0.07009241
## 4 0.04342290      4 0.5240267 0.6668256 0.06922100
## 5 0.03450324      5 0.4806038 0.6378779 0.06854061
## 6 0.01877027      7 0.4115973 0.6624756 0.08409966
## 7 0.01778685      9 0.3740568 0.7124886 0.09350971
## 8 0.01000000     11 0.3384831 0.7070176 0.09248091
```

Checking CV results in rpart

The rpart function automatically performs k-fold CV for the pruning parameter α

- To access results, append $cptable to the rpart model object:

```
tree_model1$cptable
```

```
##           CP nsplit rel error    xerror       xstd
## 1 0.31073097      0 1.0000000 1.0105895 0.09666964
## 2 0.07370105      1 0.6892690 0.7679112 0.07560215
## 3 0.04577064      2 0.6155680 0.7211540 0.07009241
## 4 0.04342290      4 0.5240267 0.6668256 0.06922100
## 5 0.03450324      5 0.4806038 0.6378779 0.06854061
## 6 0.01877027      7 0.4115973 0.6624756 0.08409966
## 7 0.01778685      9 0.3740568 0.7124886 0.09350971
## 8 0.01000000     11 0.3384831 0.7070176 0.09248091
```

- CP stands for "complexity parameter" – it's the (minimum) α that yields that tree
- nsplit is the number of times the data was split (number of leaves - 1)
- rel error is a standardized measure of MSE:

$$\text{rel error} = 1 - R^2, \quad \text{where} \quad R^2 = 1 - \frac{\text{RSS}}{\text{TSS}}$$

- xerror is cross-validated estimate of rel error
- xstd is the standard deviation in xerror based on CV

Model Summary with printcp()

The printcp function displays key model information

```
printcp(tree_model1)
```

```
##
## Regression tree:
## rpart(formula = Carbon_Sequestration_lb ~ Tree_Height + Crown_Width_EW +
##     Crown_Width_NS + Crown_Base_Height, data = my_pdxTrees)
##
## Variables actually used in tree construction:
## [1] Crown_Base_Height Crown_Width_EW    Crown_Width_NS    Tree_Height
##
## Root node error: 406713/307 = 1324.8
##
## n= 307
##
##         CP nsplit rel error  xerror     xstd
## 1 0.310731      0   1.00000 1.01059 0.096670
## 2 0.073701      1   0.68927 0.76791 0.075602
## 3 0.045771      2   0.61557 0.72115 0.070092
## 4 0.043423      4   0.52403 0.66683 0.069221
## 5 0.034503      5   0.48060 0.63788 0.068541
## 6 0.018770      7   0.41160 0.66248 0.084100
## 7 0.017787      9   0.37406 0.71249 0.093510
## 8 0.010000     11   0.33848 0.70702 0.092481
```

Model Summary with summary()

More detailed results can be accessed via summary:

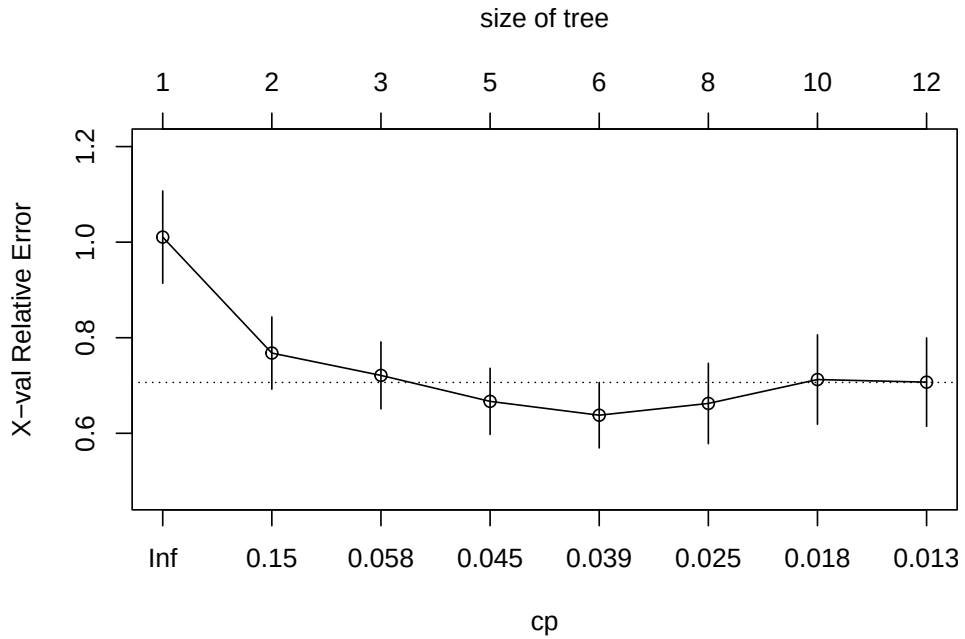
```
summary(tree_model1)
```

```
## Call:
## rpart(formula = Carbon_Sequestration_lb ~ Tree_Height + Crown_Width_EW +
##     Crown_Width_NS + Crown_Base_Height, data = my_pdxTrees)
##   n= 307
##
##           CP nsplit rel error    xerror       xstd
## 1 0.31073097      0 1.0000000 1.0105895 0.09666964
## 2 0.07370105      1 0.6892690 0.7679112 0.07560215
## 3 0.04577064      2 0.6155680 0.7211540 0.07009241
## 4 0.04342290      4 0.5240267 0.6668256 0.06922100
## 5 0.03450324      5 0.4806038 0.6378779 0.06854061
## 6 0.01877027      7 0.4115973 0.6624756 0.08409966
## 7 0.01778685      9 0.3740568 0.7124886 0.09350971
## 8 0.01000000     11 0.3384831 0.7070176 0.09248091
##
## Variable importance
##    Crown_Width_EW    Crown_Width_NS       Tree_Height Crown_Base_Height
##                38                28                24                10
##
## Node number 1: 307 observations,    complexity param=0.310731
##   mean=47.95081, MSE=1324.797
##   left son=2 (137 obs) right son=3 (170 obs)
##   Primary splits:
##       Crown_Width_EW    < 43.5 to the left,  improve=0.31073100, (0 missing)
##       Crown_Width_NS    < 49.5 to the left,  improve=0.28692940, (0 missing)
##       Tree_Height       < 78.5 to the left,  improve=0.16233240, (0 missing)
##       Crown_Base_Height < 4.5  to the left,  improve=0.05039755, (0 missing)
##   Surrogate splits:
##       Crown_Width_NS    < 43.5 to the left,  agree=0.788, adj=0.526, (0 split)
##       Tree_Height       < 45.5 to the left,  agree=0.739, adj=0.416, (0 split)
```

CV Plot with `plotcp()`

We can plot the results of cross-validation using `plotcp`:

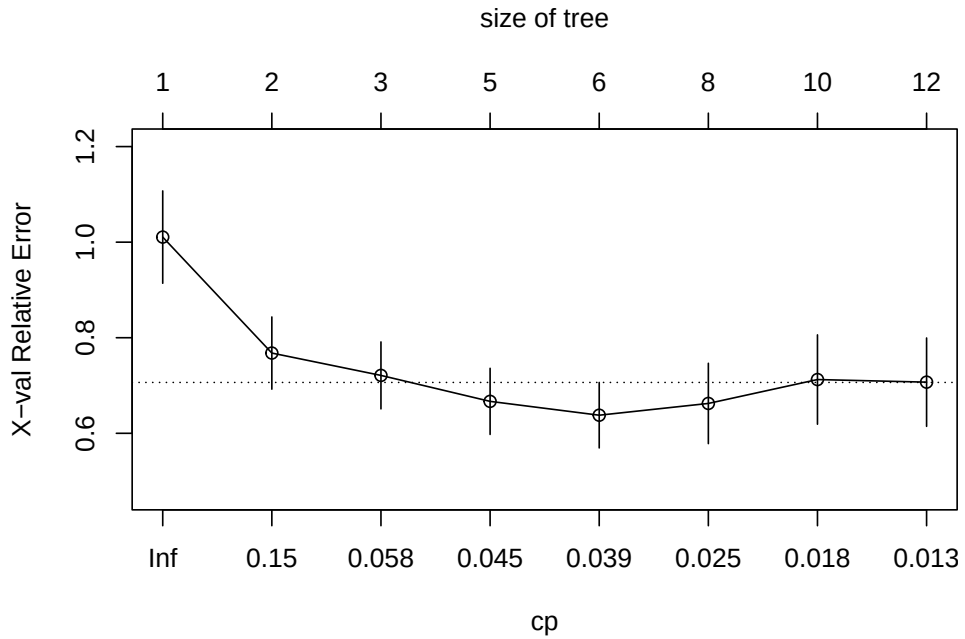
```
plotcp(tree_model1)
```



CV Plot with `plotcp()`

We can plot the results of cross-validation using `plotcp`:

```
plotcp(tree_model1)
```



Notice: The `cp` here is different than the `CP` in previous table

- But they're saying the same thing – the α that would give you the corresponding tree
- Safer to use this `cp` (NOT `CP`) to pull out the correct trees

Choosing the pruned tree with `prune()`

Based on the CV plot, the tree with 6 leaves (or 5 splits) gives the lowest error

- This tree had a $cp = 0.039$

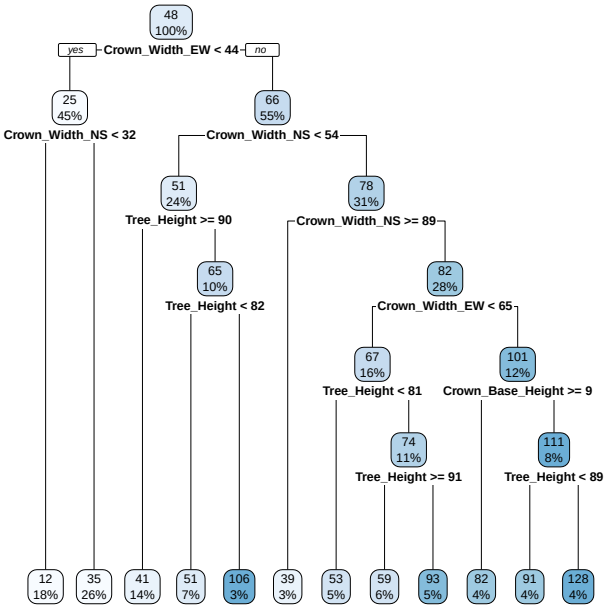
Choosing the pruned tree with `prune()`

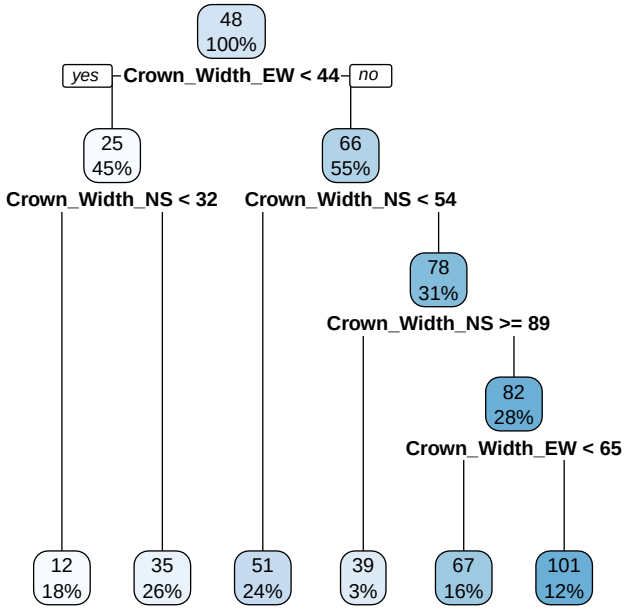
Based on the CV plot, the tree with 6 leaves (or 5 splits) gives the lowest error

- This tree had a $cp = 0.039$

We can pull out the pruned subtree using the `prune` function with a given value of `cp`

```
pruned_tree <- prune(tree_model1, cp = 0.039)
```





Making predictions with `predict()`

Can again use the `predict()` function!

```
predict(pruned_tree, my_pdxTrees_test)
```

```
##         1         2         3         4         5         6         7
##  11.69464 100.76757  67.38600  51.22000 100.76757  34.79012 100.76757   1
##         9        10
##  51.22000  34.79012
```

Test rMSE with the yardstick package

How well do models do on the test data?

- Can use the yardstick package, but need a results data frame in the correct format:

```
results <- data.frame(model = "full",
                      obs = my_pdxTrees_test$Carbon_Sequestration_lb,
                      preds = predict(tree_model1, my_pdxTrees_test))
results <- rbind(results,
                 data.frame(model = "pruned",
                      obs = my_pdxTrees_test$Carbon_Sequestration_lb,
                      preds = predict(pruned_tree, my_pdxTrees_test)))
```

Test rMSE with the yardstick package

How well do models do on the test data?

- Can use the yardstick package, but need a results data frame in the correct format:

```
results <- data.frame(model = "full",
                      obs = my_pdxTrees_test$Carbon_Sequestration_lb,
                      preds = predict(tree_model1, my_pdxTrees_test))
results <- rbind(results,
                 data.frame(model = "pruned",
                      obs = my_pdxTrees_test$Carbon_Sequestration_lb,
                      preds = predict(pruned_tree, my_pdxTrees_test)))
```

- Then use rmse() from yardstick to assess:

```
library(yardstick)
results %>% group_by(model) %>% rmse(truth = obs, estimate = preds)
```

```
## # A tibble: 2 x 4
##   model  .metric .estimator .estimate
##   <chr>  <chr>   <chr>          <dbl>
## 1 full   rmse    standard        21.1
## 2 pruned rmse    standard        19.2
```

Section 3

Recap

Recap

Today, we...

- Discussed pruning as a way of improving accuracy of tree models
- Implemented trees and cost complexity pruning in R

Next time, we will work on...

- Lab: Regression trees in R